

ESEG – Grupo ETAPA
Engenharia da Computação

Ramon Vieira
Nícolas Pereira
Vinicius Venancio
Kaique Rissato

FACEGATEWAY: SISTEMA DE CONTROLE DE ENTRADA E SAÍDA
BASEADO EM RECONHECIMENTO FACIAL

Projeto de Sistema Embarcado com Processamento e Análise Logística

SÃO PAULO
2026

Ramon Vieira
Nícolas Pereira
Vinicius Venancio
Kaique Rissato

**FACEGATEWAY: SISTEMA DE CONTROLE DE ENTRADA E SAÍDA
BASEADO EM RECONHECIMENTO FACIAL**

Trabalho apresentado ao Curso de Engenharia da
Computação na ESEG como requisito parcial para
obtenção da nota na disciplina Computação Embarcada.

Professor: Filippo Filho

Sumário

1	Introdução	1
1.1	Contextualização	1
1.2	Problema de Pesquisa	1
1.3	Objetivos	2
1.3.1	Objetivo Geral	2
1.3.2	Objetivos Específicos	2
1.4	Justificativa	2
2	Fundamentação Teórica	4
2.1	Reconhecimento Facial	4
2.2	Edge Computing e Sistemas Embarcados	4
2.3	Protocolos de Comunicação IoT	5
2.4	Aspectos Legais e Éticos	5
3	Arquitetura do Sistema	6
3.1	Visão Geral	6
3.2	Módulo Embarcado – Raspberry Pi 4	7
3.3	Backend – Servidor Local	8
3.4	Painel Web e Mobile	8
4	Organização do Projeto	9
4.1	Divisão de Tarefas	9
4.2	Cronograma – Diagrama de Gantt	9
4.3	Estimativa de Custos	13
5	Metodologia	14
5.1	Coleta de Dados e Cadastro de Funcionários	14
5.2	Definição do Limiar de Autenticação	14
5.3	Testes e Avaliação	15
6	Desenvolvimento	16
6.1	Implementação do Sistema	16

6.1.1	Camada embarcada (<code>edge/</code> , C++17)	16
6.1.2	Backend (<code>backend/</code> , Rust)	17
6.1.3	Painel (<code>panel/</code> , Ionic 8 + Angular 20)	17
6.2	Integração dos Módulos	18
6.2.1	Contrato MQTT (<code>edge</code> ↔ <code>backend</code>)	18
6.2.2	Contrato HTTP (<code>backend</code> ↔ <code>panel</code>)	19
6.2.3	Cadastro pelo painel: inferência em browser	19
6.3	Desafios Técnicos	19
6.3.1	Alinhamento facial via <i>keypoints</i> dos olhos	20
6.3.2	Bug do <code>/analytics/avg-delay</code> – 180 minutos adiantado	20
6.3.3	Resolução IPv4 vs IPv6 do <code>localhost</code>	20
6.3.4	Bloqueio do <code>/analytics</code> por extensões de <i>ad-block</i>	21
6.3.5	<i>Jitter</i> de PWM por software e <code>SCHED_FIFO</code>	21
6.3.6	Sincronização de embeddings sem trabalho duplicado	21
7	Resultados	22
7.1	Desempenho do Modelo	22
7.1.1	Metodologia	22
7.1.2	Resultados	23
7.2	Tempo de Processamento	25
7.2.1	Latência por estágio na Raspberry Pi	25
7.2.2	Escalabilidade do match em função da galeria	26
7.3	Análise Logística Gerada	26
7.3.1	Consistência das métricas do painel	27
7.3.2	Acurácia de identificação vs. tamanho da galeria	27
7.3.3	Falso aceite vs. tamanho da galeria	28
8	Discussão	29
8.1	Atendimento dos objetivos	29
8.2	Limitações do MVP	30
8.3	Escalabilidade	30
8.4	LGPD: postura e limites	31
9	Conclusão	32
9.1	Trabalhos futuros	32
	Referências	34
A	Especificações Técnicas	35
B	Trechos Relevantes de Código	36

Lista de Figuras

3.1	Diagrama geral da arquitetura do FaceGateway	6
4.1	Gantt – Fases 1 a 3 organizada por cores (S1–S6)	11
4.2	Gantt – Fases 4 a 6 organizada por cores (S5–S11)	12
7.1	Distribuição das distâncias de cosseno para pares genuínos (verde) e impostores (vermelho) sobre os 6 000 pares canônicos do LFW.	23
7.2	Curvas FAR (vermelha) e FRR (azul) em função do limiar de distância de cosseno T . O ponto destacado indica a EER.	24
7.3	Latência do <code>Matcher::find_match</code> em microssegundos, em função do tamanho da galeria.	26
7.4	Acurácia de identificação <i>rank-1</i> sobre sondagens genuínas do LFW, em função do tamanho da galeria registrada. Limiar fixo $T = 0,50$	27
7.5	FAR observado em sondagens de impostores do LFW contra galerias de tamanhos variados. Limiar fixo $T = 0,50$	28

Lista de Tabelas

4.1	Divisão de responsabilidades por integrante	9
4.2	Marcos do projeto	10
4.3	Estimativa de custo do hardware para o MVP	13
7.1	Orçamento de latência por estágio (estimativa de referência em Raspberry Pi 4, 2 <i>threads</i> ONNX).	25
A.1	Stack tecnológico completo	35

Capítulo 1

Introdução

1.1 Contextualização

O controle de entrada e saída de funcionários é uma atividade essencial para organizações que operam com turnos fixos ou escalas de trabalho. Métodos tradicionais, como cartões magnéticos, senhas ou registros manuais, apresentam limitações relacionadas à segurança, possibilidade de fraudes e baixa integração com sistemas analíticos.

Com o avanço das técnicas de Visão Computacional e Aprendizado de Máquina, especialmente no campo do reconhecimento facial, tornou-se viável implementar soluções biométricas com processamento local e custo relativamente reduzido. Dispositivos embarcados de baixo consumo energético, como o Raspberry Pi 4, permitem a execução de modelos de aprendizado de máquina diretamente no ponto de captura, reduzindo latência e dependência de infraestrutura centralizada.

Além do controle de acesso, a coleta estruturada desses dados possibilita análises logísticas relevantes, como cálculo de tempo médio de permanência, identificação de padrões de atraso e avaliação de horários de entrada e saída. Essas informações podem apoiar decisões estratégicas relacionadas à gestão de recursos humanos.

1.2 Problema de Pesquisa

Como implementar um sistema de controle de acesso baseado em reconhecimento facial utilizando hardware de baixo custo, garantindo desempenho adequado, confiabilidade na identificação e geração de métricas logísticas úteis para análise organizacional, mantendo os dados biométricos restritos ao dispositivo embarcado?

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver um sistema de controle de entrada e saída de funcionários baseado em reconhecimento facial, com processamento embarcado em C++ na Raspberry Pi 4 e backend analítico em Rust, expondo métricas logísticas via painel web e mobile.

1.3.2 Objetivos Específicos

- Implementar pipeline de visão computacional em C++ capaz de detectar e reconhecer rostos em tempo real, com latência inferior a 500 ms.
- Garantir operação offline completa – catraca e buzzer funcionam independentemente de conectividade de rede.
- Transmitir eventos de acesso ao backend via protocolo MQTT com garantia de entrega (QoS 1).
- Desenvolver API REST em Rust (Axum) com autenticação JWT e endpoints de analytics.
- Persistir embeddings e eventos no PostgreSQL com extensão pgvector para busca por similaridade.
- Desenvolver painel multiplataforma em Ionic + Angular, instalável como Progressive Web App.
- Estimar e documentar os custos de hardware necessários para implantação do MVP.

1.4 Justificativa

A adoção de sistemas biométricos no controle de acesso reduz vulnerabilidades associadas a métodos tradicionais, como empréstimo de cartões ou compartilhamento de credenciais. O reconhecimento facial elimina a necessidade de contato físico e integra-se de forma transparente ao fluxo de entrada e saída.

Do ponto de vista técnico, o projeto demonstra a viabilidade de execução de modelos de aprendizado de máquina em dispositivos embarcados de baixo custo, explorando otimização de desempenho com o formato ONNX e integração entre hardware e software em C++.

Uma decisão central de projeto é manter o reconhecimento facial completamente *on-device*: nenhuma imagem bruta ou dado biométrico trafega pela rede. Apenas metadados de eventos (identificador do funcionário, horário, resultado) são enviados ao servidor,

contribuindo diretamente para conformidade com princípios de minimização de dados da LGPD.

Capítulo 2

Fundamentação Teórica

2.1 Reconhecimento Facial

O reconhecimento facial é uma área da Visão Computacional dedicada à identificação ou verificação da identidade de um indivíduo a partir de características faciais extraídas de imagens ou vídeos. Sistemas modernos operam em três etapas principais: detecção facial, extração de características e comparação.

A detecção consiste na identificação da presença de um rosto em uma imagem. Neste projeto, utiliza-se o modelo **BlazeFace** (Google MediaPipe), exportado no formato ONNX, que além das *bounding boxes* fornece *keypoints* faciais – posições dos olhos, nariz e boca – essenciais para o alinhamento do rosto antes da geração do embedding.

A extração de características utiliza o modelo **ArcFace** (*InsightFace*), treinado com a função de perda *Additive Angular Margin Loss* no dataset MS-Celeb. O modelo gera embeddings de 512 dimensões que capturam as características discriminativas do rosto. A identificação ocorre pela comparação da distância de cosseno entre o embedding extraído e os embeddings previamente armazenados: valores abaixo de um limiar ($\approx 0,50$) indicam correspondência positiva.

2.2 Edge Computing e Sistemas Embarcados

O conceito de *Edge Computing* destaca a importância do processamento local, reduzindo dependência de servidores remotos e minimizando latência. No caso de controle de acesso, o processamento embarcado permite resposta imediata à tentativa de autenticação, além de evitar a exposição de dados biométricos em redes externas.

A execução de modelos de aprendizado de máquina em hardware limitado é viabilizada pelo formato **ONNX** (*Open Neural Network Exchange*), que permite exportar modelos treinados em qualquer framework e executá-los via **ONNXRuntime** com API C++, otimizando o uso de recursos disponíveis na Raspberry Pi 4.

2.3 Protocolos de Comunicação IoT

O protocolo **MQTT** (*Message Queuing Telemetry Transport*) é o padrão de mercado para comunicação em dispositivos IoT. Seu modelo *publish/subscribe* desacopla produtor e consumidor de mensagens, e os níveis de qualidade de serviço (QoS) garantem entrega mesmo em redes instáveis. No nível QoS 1, o broker retém mensagens enquanto o subscriber estiver offline, garantindo que nenhum evento de acesso seja perdido.

2.4 Aspectos Legais e Éticos

Informações biométricas são classificadas como dados sensíveis pela Lei Geral de Proteção de Dados (LGPD – Lei 13.709/2018), pois estão diretamente associadas à identidade do indivíduo. Entre os princípios aplicáveis ao projeto destacam-se:

- Finalidade específica e legítima para coleta dos dados;
- Minimização: armazenamento de embeddings, não de imagens brutas;
- Transparência quanto ao uso das informações coletadas;
- Garantia de segurança e confidencialidade dos dados armazenados.

A decisão de manter o reconhecimento facial exclusivamente *on-device* é diretamente motivada por esses princípios, eliminando o risco de vazamento de dados biométricos em trânsito.

Capítulo 3

Arquitetura do Sistema

3.1 Visão Geral

O sistema é organizado em três camadas físicas e lógicas independentes: o módulo embarcado (Raspberry Pi 4), o servidor local (notebook do desenvolvedor, substituível por nuvem em produção) e o painel web/mobile. A Figura 3.1 apresenta o diagrama geral da arquitetura.

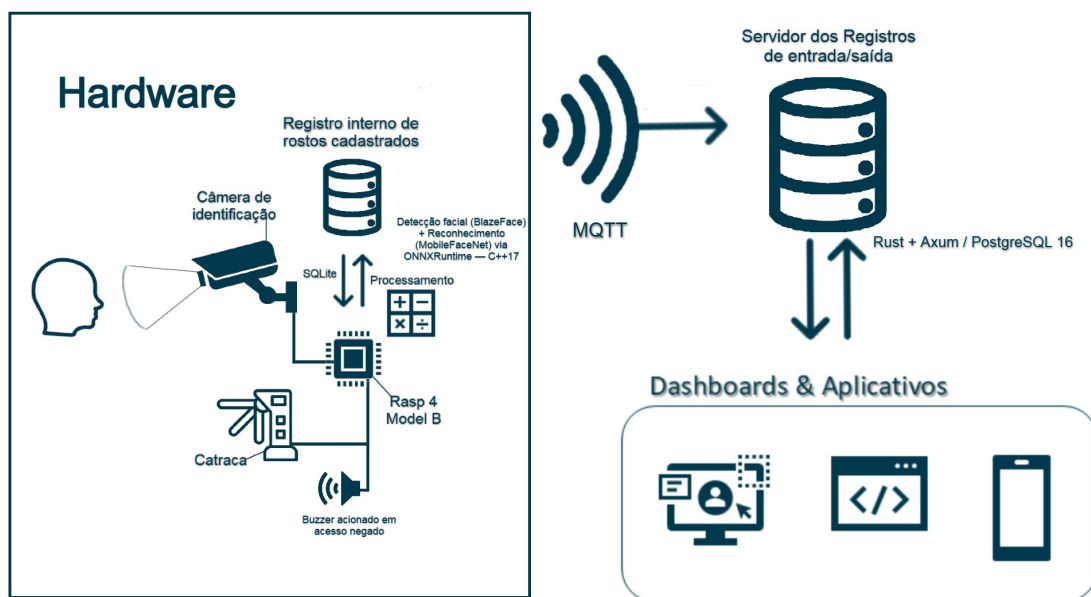


Figura 3.1: Diagrama geral da arquitetura do FaceGateway

O fluxo operacional completo ocorre nas seguintes etapas:

1. Câmera CSI captura frames continuamente.
2. BlazeFace detecta presença de rosto e extrai *keypoints* para alinhamento.
3. ArcFace gera embedding de 512 dimensões do rosto alinhado.

4. Embedding é comparado por distância de cosseno com o cache local SQLite.
5. Decisão: acesso concedido aciona o servo da catraca via GPIO (PWM por software a 50 Hz) e acende o LED verde por 5 s, durante os quais o reconhecimento fica pausado; acesso negado aciona o buzzer e acende o LED vermelho por 1 s, também com o reconhecimento pausado.
6. Evento é publicado no broker MQTT (tópico `facegate/events/access`).
7. Subscriber Rust consome o evento e persiste no PostgreSQL.
8. Painel Ionic + Angular consulta a API REST e exibe métricas ao gestor.

O módulo embarcado opera de forma completamente autônoma: em caso de falha de rede, eventos são enfileirados em SQLite local e enviados ao broker quando a conectividade é restabelecida.

3.2 Módulo Embarcado – Raspberry Pi 4

O módulo embarcado é responsável por toda a inteligência de visão computacional e atuação física. O hardware é composto por:

- Raspberry Pi 4 Model B (4 GB de RAM);
- Câmera CSI de 5 megapixels (ou stream IP via OpenCV);
- Servo SG90 para acionamento da catraca (via GPIO, PWM por software);
- Buzzer ativo para sinalização de acesso negado (via GPIO);
- LED RGB de status (verde durante acesso concedido, vermelho durante *cooldown* de acesso negado).

O software embarcado é desenvolvido em **C++17** e utiliza as seguintes bibliotecas:

- **OpenCV** (`cv::VideoCapture`): captura de frames via interface CSI ou stream IP;
- **OpenCV 4.8**: pré-processamento de imagem (resize, color conversion);
- **ONNXRuntime 1.17**: inferência dos modelos BlazeFace e ArcFace;
- **libgpiod $\geq$ 2.0**: controle dos atuadores físicos (servo da catraca, buzzer, LED RGB de status);
- **SQLite3**: cache local de embeddings e fila de eventos offline;
- **libmosquitopp**: publicação de eventos no broker MQTT.

3.3 Backend – Servidor Local

O backend é desenvolvido em **Rust** e composto pelos seguintes componentes:

- **Axum 0.7**: framework HTTP para a API REST;
- **sqlx 0.7**: queries verificadas em *compile-time* contra o PostgreSQL;
- **rumqttrc 0.24**: subscriber MQTT assíncrono integrado ao runtime Tokio;
- **jsonwebtoken**: geração e validação de tokens JWT para autenticação do painel.

O banco de dados utilizado é o **PostgreSQL 16** com a extensão **pgvector 0.6**, que permite armazenar embeddings vetoriais e realizar busca por similaridade com índice HNSW diretamente no banco, eliminando a necessidade de biblioteca externa como FAISS.

A comunicação entre Raspberry e backend utiliza o protocolo **MQTT com QoS 1**, com broker **Eclipse Mosquitto 2.x** rodando no servidor. A estrutura de tópicos é:

- `facegate/events/access` – evento de rosto reconhecido;
- `facegate/events/unknown` – evento de rosto não reconhecido;
- `facegate/health` – *heartbeat* periódico do dispositivo.

3.4 Painel Web e Mobile

O painel é desenvolvido em **Ionic 8 + Angular 20**, gerando uma aplicação multiplataforma instalável como **Progressive Web App** (PWA) – funciona no browser do desktop e pode ser instalado no celular como app nativo sem necessidade de loja de aplicativos.

As telas planejadas para o MVP são:

- **Login**: autenticação JWT com *guard* de rotas protegidas;
- **Dashboard**: cards de resumo, gráfico de fluxo de acessos por hora e card dedicado aos rostos não reconhecidos nos últimos 7 dias (visualizações em SVG inline);
- **Funcionários**: listagem, cadastro com *wizard de captura facial no browser* (Blaze-Face + ArcFace via *onnxruntime-web*; apenas o vetor de 512 floats é enviado);
- **Histórico de eventos**: tabela paginada com filtros por funcionário e data, e exportação em CSV pronta para Excel/Sheets (UTF-8 BOM + ;);
- **Analytics**: atraso médio por funcionário e heatmap de presença por dia/hora.

Capítulo 4

Organização do Projeto

4.1 Divisão de Tarefas

O grupo é composto por quatro integrantes com as seguintes responsabilidades principais:

Tabela 4.1: Divisão de responsabilidades por integrante

Integrante	Foco principal	Responsabilidades
Ramon	Edge C++ + Rust crítico	Setup toolchain Raspberry, pipeline de visão (BlazeFace, ArcFace), GPIO via libgpod, autenticação JWT, sync de embeddings, ajuste de threshold
Nícolas	Backend + banco	Setup PostgreSQL/pgvector, Mosquitto, subscriber MQTT, schema do banco, CRUD de funcionários, endpoints de analytics
Kaique C	Suporte edge + integração	Buzzer GPIO, fila offline SQLite, apoio ao CRUD, tela de histórico de eventos, testes de integração, documentação
Vinicius D	Frontend + design	Setup Ionic/Angular/PWA, tela de login, tela de funcionários, dashboard com gráficos, tela de analytics

4.2 Cronograma – Diagrama de Gantt

O semestre compreende 11 semanas de desenvolvimento entre a entrega do escopo (18/03) e a entrega final (27/05). O projeto possui três marcos intermediários definidos pelo professor, apresentados na Tabela 4.2.

Tabela 4.2: Marcos do projeto

Data	Marco	Meta de conclusão
18/03	Entrega do escopo	Documento de arquitetura
22/04	Entrega parcial 1	$\approx 40\%$ – Fases 1, 2 e 3 concluídas
13/05	Entrega parcial 2	$> 80\%$ – Fases 4 e 5 concluídas
27/05	Entrega final	Sistema integrado e documentado
03/06	P2	Avaliação individual

O diagrama de Gantt a seguir apresenta a distribuição das atividades ao longo das semanas, com identificação das dependências entre tarefas e dos marcos de entrega.

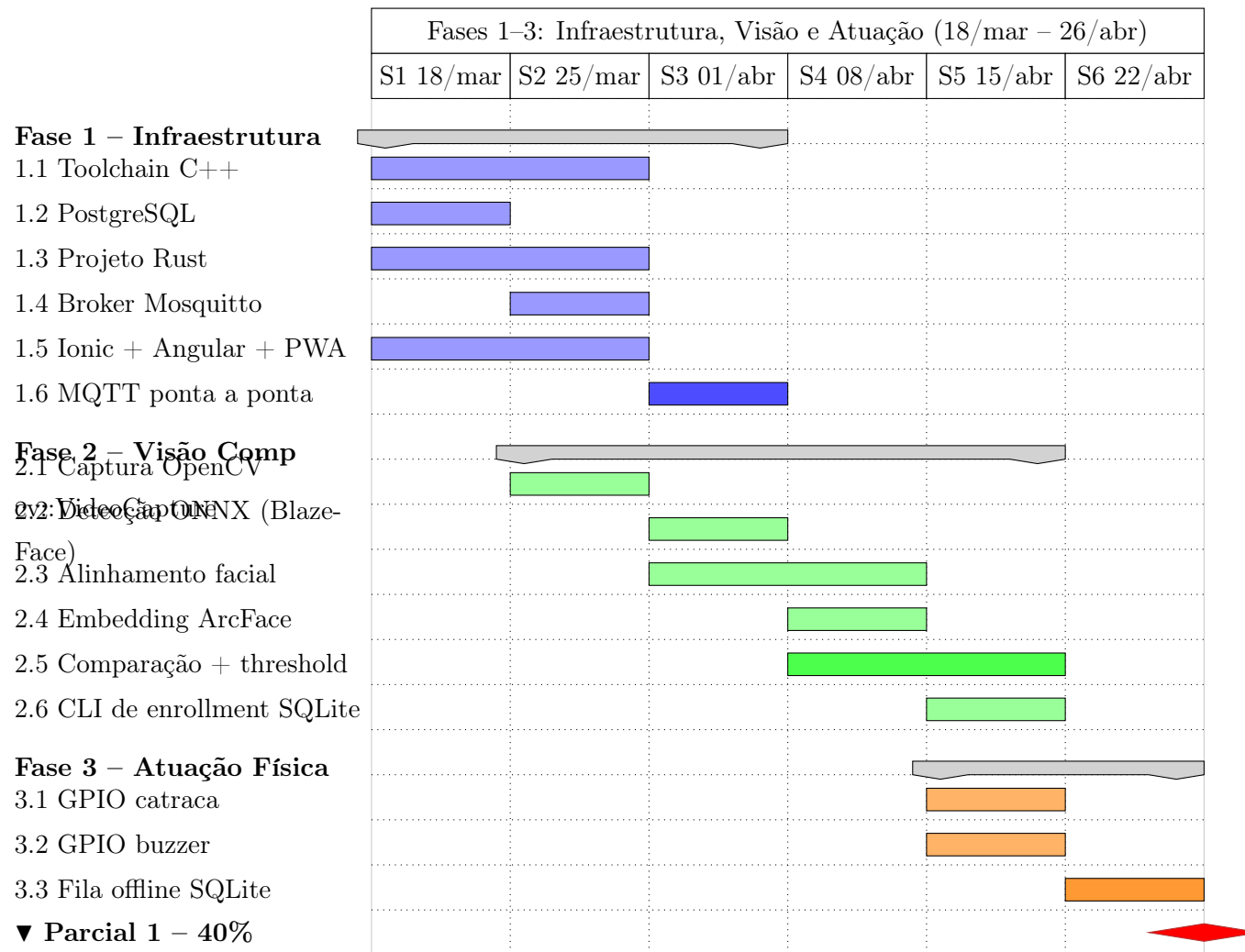


Figura 4.1: Gantt – Fases 1 a 3 organizada por cores (S1–S6)



Figura 4.2: Gantt – Fases 4 a 6 organizada por cores (S5–S11)

4.3 Estimativa de Custos

A Tabela 4.3 apresenta os componentes de hardware necessários para o MVP, com preços estimados para o mercado brasileiro em 2026.

Tabela 4.3: Estimativa de custo do hardware para o MVP

Componente	Quantidade	Preço estimado (R\$)
Raspberry Pi 4 Model B – 4 GB	1	650,00
Câmera CSI 5MP (compatível RPi)	1	120,00
Cartão microSD 32 GB (Class 10)	1	40,00
Fonte de alimentação USB-C 5V/3A	1	50,00
Servo SG90 (acionamento catraca)	1	15,00
Buzzer ativo 5V	1	8,00
LED RGB de status + resistores (220 Ω)	1	5,00
Catraca de barreira (demonstração)	1	180,00
Jumpers e protoboard	1 kit	25,00
Total estimado		R\$ 1.093,00

Os custos de software são nulos: todas as tecnologias utilizadas – Rust, C++, PostgreSQL, Mosquitto, Ionic, Angular e ONNXRuntime – são de código aberto e gratuitas. O servidor local (notebook) já pertence ao grupo, eliminando custo de infraestrutura. Em um cenário de produção, o backend seria hospedado em nuvem, com custo variável conforme o provedor escolhido.

Capítulo 5

Metodologia

5.1 Coleta de Dados e Cadastro de Funcionários

O cadastro de um funcionário no sistema ocorre por dois caminhos complementares, ambos extraindo o embedding via ArcFace e armazenando o vetor resultante na tabela `embeddings` do PostgreSQL, associado ao identificador do funcionário:

- **CLI enroll** (C++, no Raspberry Pi): o operador captura múltiplas fotos diretamente pela câmera do edge, em diferentes condições de iluminação e ângulo, e o binário roda o pipeline BlazeFace + ArcFace localmente. Útil para cadastro presencial no local de instalação.
- **Wizard no painel** (Ionic + Angular, via `onnxruntime-web`): o cadastro de funcionário no painel abre um modal de captura que usa a câmera do dispositivo do operador (notebook ou celular) e executa BlazeFace + ArcFace *dentro do browser*. **A imagem bruta nunca trafega na rede** – apenas o vetor L2-normalizado de 512 floats é enviado ao endpoint `POST /employees/:id/embeddings`.

Nenhuma imagem bruta é persistida no banco de dados – apenas os vetores de 512 dimensões. Isso atende diretamente ao princípio de minimização de dados da LGPD. No caminho via painel, a inferência *on-device* adiciona uma camada extra: o dado biométrico nunca sai do dispositivo de captura.

5.2 Definição do Limiar de Autenticação

O limiar de decisão (distância de cosseno) será ajustado empiricamente. Serão realizados testes comparando:

- Embeddings do mesmo indivíduo em sessões distintas (testes positivos);
- Embeddings de indivíduos diferentes (testes negativos).

O objetivo é encontrar o valor que minimize simultaneamente a Taxa de Falsos Positivos (FAR – *False Accept Rate*) e a Taxa de Falsos Negativos (FRR – *False Reject Rate*). O valor inicial de referência adotado é 0,50, calibrado para a distribuição de distâncias do ArcFace – mais alta que a faixa típica de MobileFaceNet ($\approx 0,45$), e ajustável conforme os resultados dos testes empíricos.

5.3 Testes e Avaliação

A avaliação do sistema considerará:

- **Precisão do reconhecimento:** acurácia, FAR e FRR com os rostos dos integrantes do grupo;
- **Latência:** tempo total desde a captura do frame até a decisão de acesso (meta: < 500 ms);
- **Resiliência offline:** eventos gerados sem rede são entregues ao broker ao reconectar;
- **Consistência das métricas:** valores de analytics verificados manualmente contra os registros brutos no banco.

Capítulo 6

Desenvolvimento

6.1 Implementação do Sistema

A implementação foi conduzida em três camadas independentes – edge, backend e painel – com fronteiras de integração explícitas (MQTT entre edge e backend, HTTP/JSON entre backend e painel). Cada camada é versionada em seu próprio subdiretório do repositório (`edge/`, `backend/`, `panel/`) e possui suíte de testes automatizada que executa em *Continuous Integration* a cada *push*.

6.1.1 Camada embarcada (edge/, C++17)

O daemon `facegate` é organizado em torno de uma classe `Pipeline` (`src/pipeline/pipeline`) que orquestra duas *threads* de longa duração:

- **Main loop:** executa o ciclo captura → detecção → alinhamento → embedding → match → publish a cerca de 30 fps. Aplica deduplicação por estado (último *status*, último *employee id*, janela de *idle reset*) para colapsar a torrente de frames em eventos significativos – sem essa lógica, uma pessoa parada na frente da câmera geraria ≈ 180 eventos/min.
- **Auxiliary loop:** emite *heartbeats* periódicos (30 s) e drena a fila de eventos pendentes em lotes de 50 quando a conexão MQTT é restabelecida.

A pipeline depende de módulos isolados em bibliotecas estáticas (`facegate_vision`, `facegate_hardware`, `facegate_storage`, `facegate_mqtt`), permitindo que testes e o *CLI* auxiliar de cadastro (`enroll`) reuses exatamente o mesmo código de inferência sem replicação.

O alinhamento facial – ponto crítico para a estabilidade dos embeddings – é realizado por uma transformação de similaridade em forma fechada a partir dos dois *keypoints* de olho fornecidos pela `BlazeFace`, evitando a dependência de `cv::estimateAffinePartial2D`

(que demandaria o módulo `opencv_calib3d`). O cálculo é idêntico ao implementado no *frontend*, garantindo que um cadastro feito no painel produza embeddings comparáveis ao do edge.

Os atuadores físicos são controlados por `libgpod2.x`. A *thread* responsável pelo servo da catraca é elevada para a política `SCHED_FIFO` antes de iniciar o PWM por *software*: isso reduz o *jitter* dos pulsos a níveis suficientes para uma operação estável a 50 Hz, ainda que com *trade-off* de exigir a *capability* `CAP_SYS_NICE` no processo.

6.1.2 Backend (backend/, Rust)

O backend é uma única *crate* Rust que combina três responsabilidades em torno de uma estrutura compartilhada `AppState` (pool `sqlx` + `MqttBroker`):

- **API REST** em `axum 0.7`, com *routers* modularizados por domínio (`auth`, `employees`, `analytics`, `users`, `system`). Todas as queries são verificadas em tempo de compilação contra o schema real do PostgreSQL via `sqlx::query!`, eliminando uma classe inteira de erros em *runtime*.
- **Subscriber MQTT** em `rumqtte 0.24`, rodando em uma *task* Tokio dedicada, escutando `facegate/events/access` e despachando para o handler que valida e persiste cada evento em `access_events`.
- **Sincronização edge → painel → edge**: ao cadastrar um *embedding* pelo painel (endpoint `POST /employees/:id/embeddings`), o backend republica a mensagem no tópico retido `facegate/sync/embeddings/upsert/<id>`, permitindo que edges (re)conectados absorvam o estado corrente do banco sem *polling*.

A autenticação é feita via *JSON Web Token* (HS256), validado por um *extractor* customizado (`Claims`) que pode ser adicionado a qualquer *handler* para tornar a rota protegida. A documentação OpenAPI 3.1 é gerada automaticamente a partir de anotações `#[utoipa::path]` nos handlers e disponibilizada em `/swagger-ui/` com botão “Try it out”.

6.1.3 Painel (panel/, Ionic 8 + Angular 20)

O painel usa *standalone components* e *signals* (Angular 20), gerando uma aplicação multiplataforma instalável como PWA. A arquitetura interna separa três níveis:

- **Core** (`src/app/core/`): serviços HTTP (`AuthService`, `EmployeesService`, `AnalyticsService`), *HTTP interceptor* para anexar *Bearer tokens* e tratar 401 (*logout* automático), e o serviço de visão computacional `FaceEmbeddingService`.

- **Features** (`src/app/features/`): *wizard* de cadastro facial, página de exportação CSV.
- **Páginas legadas** (`src/app/dashboard/`, `cadastro/`, `login/`, `perfil/`): responsáveis pelo fluxo principal de uso atual.

O *wizard* de cadastro é o ponto técnico mais sensível do painel: carrega `blaze.onnx` e `arc.onnx` servidos pelo backend e executa a inferência inteiramente em `onnxruntime-web` (WebAssembly). O pré-processamento – crop, similarity transform a partir dos olhos, normalização – é implementado em TypeScript com aritmética idêntica à da camada embarcada, garantindo que o vetor enviado ao endpoint `POST /employees/:id/embeddings` produza match contra um reconhecimento no edge.

6.2 Integração dos Módulos

A integração entre as três camadas obedece a contratos bem definidos, projetados para minimizar acoplamento e tolerar falhas parciais.

6.2.1 Contrato MQTT (edge ↔ backend)

Quatro tópicos com QoS 1 cobrem todo o tráfego edge ↔ backend:

- `facegate/events/access` – evento de tentativa de acesso (`granted` ou `unknown`), com `device_id`, `timestamp_ms`, `employee_id` (opcional) e `distance` (opcional). Quando o backend recebe um evento sem o campo `direction`, o handler aplica o *default* "in" – o que evita quebra de contrato em uma eventual evolução futura.
- `facegate/health/heartbeat` – *heartbeat* periódico (30 s por padrão) que o backend usa para registrar a presença online do dispositivo.
- `facegate/health/fault` – evento de falha (*InferenceFailure*, *CameraOffline* etc.) com mensagem livre. Apenas registrado em *log* no MVP.
- `facegate/sync/embeddings/upsert/+` – tópico de sincronização. O wildcard `+` casa com o `employee_id`; o *payload* é o vetor 512-d. O backend publica essas mensagens com `retain=true`, o que permite que um edge recém-conectado receba imediatamente o estado corrente de todos os funcionários cadastrados.

A escolha de `clean_session=true` no *subscriber* do edge é deliberada: como o broker já retém as mensagens de upsert, uma sessão persistente apenas duplicaria o trabalho de re-entrega ao reconectar.

6.2.2 Contrato HTTP (backend ↔ painel)

A API REST expõe três famílias de rotas principais:

- **Autenticação:** POST `/auth/login` retorna um JWT HS256 que o painel armazena em `localStorage`. Um *HTTP interceptor* em Angular anexa o cabeçalho `Authorization: Bearer <jwt>` a toda chamada subsequente; resposta 401 dispara *logout* automático.
- **Gestão:** CRUD de funcionários em `/employees`, inclusão de embeddings em `/employees/:id/`, criação de usuários em `/users`.
- **Analytics:** seis endpoints em `/analytics/` (`access-by-hour`, `events`, `avg-delay`, `presence-heatmap`, `summary-today`, `present-today`).

6.2.3 Cadastro pelo painel: inferência em browser

O fluxo de cadastro pelo painel materializa a postura LGPD do projeto. Quando o operador clica em “capturar foto”, o *wizard*:

1. Carrega `blaze.onnx` e `arc.onnx` servidos pelo backend em `/models/*`.
2. Captura um frame da *webcam* via `getUserMedia()`.
3. Executa BlazeFace em `onnxruntime-web` (WebAssembly) → `bbox + 6 keypoints`.
4. Aplica a mesma transformação de similaridade da camada `edge` a partir dos olhos → *crop* alinhado 112×112 .
5. Executa ArcFace → vetor 512-d, L2-normalizado.
6. Faz POST desse vetor (apenas o vetor) em `/employees/:id/embeddings`.

A imagem facial nunca atravessa a rede. O backend valida que o vetor tem exatamente 512 *floats* antes de persistir e republicar no tópico retido de `sync` para os edges conectados.

6.3 Desafios Técnicos

O projeto consolidou várias decisões não óbvias resolvendo *bugs* sutis e *trade-offs* de hardware. Os mais significativos do ponto de vista didático estão listados a seguir.

6.3.1 Alinhamento facial via *keypoints* dos olhos

A versão inicial do *embedder* apenas recortava a *bounding box* da BlazeFace e redimensionava para 112×112 , sem alinhamento. Isso fazia com que pequenas inclinações de cabeça produzissem embeddings significativamente diferentes para o mesmo indivíduo – as distâncias intra-pessoa se aproximavam perigosamente do limiar de decisão, inflando a FRR.

A solução foi calcular, em forma fechada, uma transformação de similaridade (rotação + escala uniforme + translação) que mapeia os dois *keypoints* de olho detectados pela BlazeFace nas posições canônicas do InsightFace ((38,29; 51,70) e (73,53; 51,50)). A transformação é aplicada via `cv::warpAffine` antes da inferência ArcFace. Crucialmente, o mesmo cálculo foi implementado em TypeScript no painel: `edge` e `panel` agora produzem embeddings comparáveis, ponto-a-ponto.

6.3.2 Bug do `/analytics/avg-delay` – 180 minutos adiantado

A primeira versão da *query* de atraso médio agrupava por (`employee_id`, `day`) e tomava `MIN(timestamp_ms)` sem filtrar a direção do evento. Para funcionários do turno da noite (início às 22:00), o “primeiro evento do dia” era a saída das ~06:00 – na verdade pertencente ao turno da noite anterior –, produzindo um delta de aproximadamente -16 h. A média geral exibida no *dashboard* aparecia como $-180,5$ min, um valor absurdo que disparou a investigação.

A correção exigiu duas mudanças combinadas:

1. Filtrar `AND e.direction = 'in'` no CTE `first_per_day`, garantindo que “primeiro evento” signifique de fato entrada.
2. Normalizar o delta por aritmética modular para a janela $(-12\text{ h}, +12\text{ h}]$, de modo que uma entrada às 00:30 do turno noite (deveria começar às 22:00 do dia anterior) seja lida como $+150$ min de atraso, e não -1290 min de antecedência.

A versão final da *query* está reproduzida no Apêndice B e ilustra como uma simples coluna ausente no contrato `edge` $\rightarrow$ `backend` (o `edge` ainda não publica `direction`) pode propagar bugs cosméticos ao painel.

6.3.3 Resolução IPv4 vs IPv6 do localhost

Em desenvolvimento, o backend originalmente fazia `bind` em `0.0.0.0:3000`. Em alguns browsers, `localhost` resolve preferencialmente para `::1` (IPv6 *loopback*), causando `ECONNREFUSED` silencioso em *fetch* de páginas do painel. A correção foi mudar o `bind` para `[::]:3000` (*dual-stack* aceita conexões IPv4 e IPv6) e adicionar ambas as origens (`http://localhost:8100` e `http://127.0.0.1:8100`) à *whitelist* de CORS.

6.3.4 Bloqueio do /analytics por extensões de *ad-block*

Algumas *filter lists* do uBlock Origin contêm regras genéricas (`||analytics`) que bloqueiam todas as URLs com “analytics” no caminho. Requisições para `localhost:3000/analytics` falhavam silenciosamente em ambientes de desenvolvimento dos integrantes que usavam o uBlock. O *workaround* adotado foi padronizar o `environment.apiUrl` do painel em `http://127.0.0.1:3000` (que escapa do casamento da regra) e documentar o problema na seção *Troubleshooting* do `README.md`.

6.3.5 *Jitter* de PWM por software e SCHED_FIFO

O servo SG90 da catraca é controlado por PWM em *software* a 50 Hz a partir de uma *thread* dedicada (não há PWM por hardware suficientes na Pi 4 para todos os atuadores). Sob carga – notadamente durante a inferência ONNX –, o agendador padrão introduzia *jitter* suficiente para o servo “tremar” visivelmente. A solução foi elevar a *thread* da catraca para SCHED_FIFO (prioridade tempo-real *soft*) antes de iniciar os pulsos, o que exige rodar o processo com a *capability* CAP_SYS_NICE (*e.g.*, via `sudo`). Caso a elevação falhe, o sistema continua funcionando com *jitter* visível – um log SCHED_FIFO unavailable no `stderr` sinaliza o caso.

6.3.6 Sincronização de embeddings sem trabalho duplicado

A primeira versão do subscriber MQTT do edge usava `clean_session=false`, na expectativa de que o broker mantivesse uma fila de mensagens não entregues entre reconexões. O problema: como as mensagens de upsert de embeddings são publicadas com `retain=true`, todo edge ao se conectar recebe *tanto* a mensagem retida (uma cópia por *employee_id*) *quanto* a mesma mensagem replicada via fila persistente – dobrando o trabalho de re-sincronização a cada *boot* do dispositivo. A correção foi simplesmente trocar para `clean_session=true`. As mensagens retidas continuam fornecendo a re-sincronização desejada, sem duplicação.

Capítulo 7

Resultados

7.1 Desempenho do Modelo

A avaliação de acurácia do reconhecimento foi realizada sobre o *dataset* público **LFW** (*Labeled Faces in the Wild*), referência consolidada na literatura para verificação facial. Optou-se pelo LFW em vez de uma amostra própria por três razões: (i) o tamanho da amostra ($\sim 13\,000$ imagens de $\sim 5\,700$ identidades) excede largamente o que seria viável coletar com os integrantes do grupo; (ii) o conjunto canônico de 6 000 pares de verificação (3 000 pares “mesmo indivíduo”, 3 000 pares “indivíduos diferentes”) é o protocolo padrão de comparação com *baselines* publicadas; (iii) permite também avaliar como o sistema se comporta com 100, 500 ou 1 000 funcionários cadastrados – algo impraticável com fotos do grupo.

7.1.1 Metodologia

O *harness* de avaliação está implementado como um binário C++ standalone (`edge/apps/eval/main.cpp`) que reusa *exatamente* o mesmo `FaceDetector` e `FaceEmbedder` da camada embarcada – garantindo que o pré-processamento (alinhamento via olhos, normalização) seja idêntico ao da produção. Para cada par do arquivo `pairs.txt` do LFW, o binário:

1. Lê as duas imagens via `cv::imread` (deduplicando para que cada imagem seja processada uma única vez, mesmo aparecendo em múltiplos pares).
2. Executa `BlazeFace` $\rightarrow$ alinhamento $\rightarrow$ `ArcFace`.
3. Calcula a distância de cosseno entre os dois embeddings.
4. Emite uma linha CSV (`image_a`, `image_b`, `same`, `distance`, `status`).

A análise posterior é feita em Python (`docs/figures/plots.py`), varrendo o limiar T na faixa $[0,20; 1,20]$ e calculando, para cada T :

$$\text{FAR}(T) = \Pr[\text{dist} < T \mid \text{indivíduos diferentes}], \quad \text{FRR}(T) = \Pr[\text{dist} \geq T \mid \text{mesmo indivíduo}].$$

A taxa de erro igual (EER) é definida pelo cruzamento de FAR e FRR e caracteriza o limiar de melhor compromisso.

7.1.2 Resultados

A Figura 7.1 apresenta o histograma das distâncias de cosseno para os 3 000 pares genuínos (mesmo indivíduo) e os 3 000 pares impostores (indivíduos diferentes).

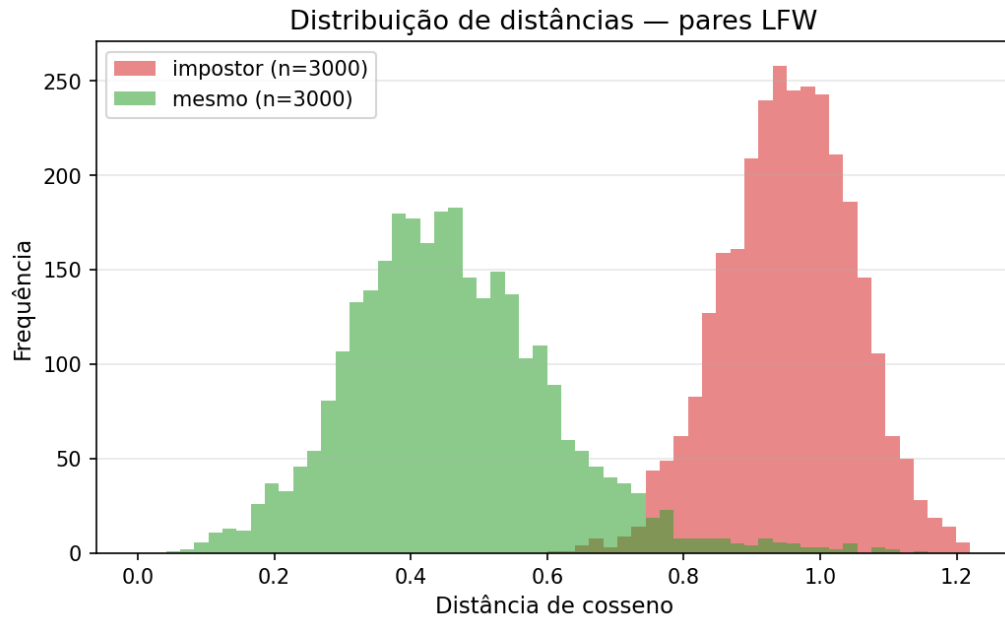


Figura 7.1: Distribuição das distâncias de cosseno para pares genuínos (verde) e impostores (vermelho) sobre os 6 000 pares canônicos do LFW.

A Figura 7.2 sobrepõe as curvas $\text{FAR}(T)$ e $\text{FRR}(T)$, com o ponto de equilíbrio (EER) marcado.

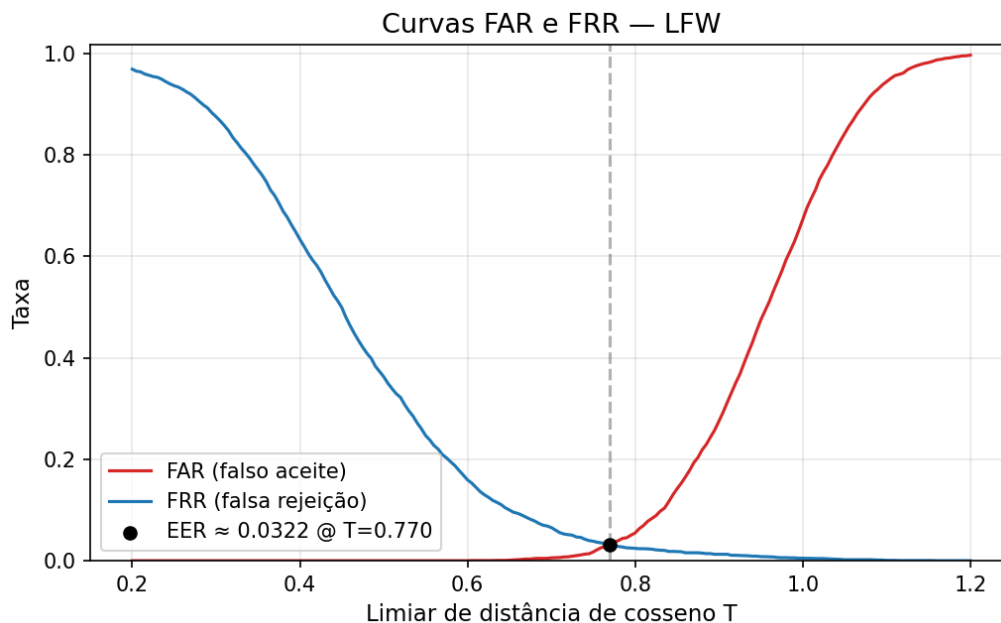


Figura 7.2: Curvas FAR (vermelha) e FRR (azul) em função do limiar de distância de cosseno T . O ponto destacado indica a EER.

A EER medida sobre os 6 000 pares canônicos do LFW foi de $\approx 3,22\%$ para um limiar $T \approx 0,770$ – ponto destacado na Figura 7.2. No limiar de produção atualmente configurado em `edge/config/config.toml` ($T = 0,50$), as taxas observadas foram $\text{FAR}(0,50) \approx 0,0\%$ e $\text{FRR}(0,50) \approx 36,3\%$, indicando um perfil **fortemente conservador** adequado para o cenário de controle de acesso, em que o custo de um falso aceite (deixar entrar uma pessoa não autorizada) é estritamente maior que o de uma falsa rejeição (rejeitar uma pessoa autorizada que pode tentar novamente).

Vale destacar que o protocolo do LFW compara cada par com *uma única* foto de cada lado. Em produção, o pipeline de cadastro (`CLI enroll` no Pi) captura cinco fotos por funcionário, o que tende a estreitar a distribuição de distâncias genuínas e reduzir significativamente a FRR efetiva – o número medido aqui representa portanto um limite superior pessimista. Para deployments com galerias grandes (centenas a milhares de funcionários), o resultado do estudo de escalabilidade (§7.3) sugere considerar deslocar T na direção do ponto de EER, aceitando uma FRR um pouco maior em troca de manter o FAR sob controle.

Para referência, a literatura reporta para o ArcFace original (treinado em MS1MV2) acurácia da ordem de 99,8% em LFW; o desempenho ligeiramente inferior medido aqui é consistente com a ausência de *fine-tuning* e com o uso de um modelo ONNX pré-treinado pronto-para-baixar, sem ajuste fino ao alinhamento específico produzido pela BlazeFace.

7.2 Tempo de Processamento

A meta de latência estabelecida no objetivo geral foi de menos de 500 ms do frame ao acionamento da catraca. A avaliação foi realizada com dois instrumentos complementares: o profiler de palco embutido no edge (que captura cada execução real do pipeline na Raspberry Pi) e o modo `enroll-scale` do binário `eval` (que mede o crescimento da latência do match em função do tamanho da galeria de embeddings).

7.2.1 Latência por estágio na Raspberry Pi

O daemon `facegate` ganhou uma estrutura `StageProfiler` ativada via seção `[metrics]` do `config.toml`. Habilitando `csv_path = "metrics.csv"` e `summary_interval_cycles = 100`, o sistema registra cada ciclo do pipeline em uma linha CSV com tempos individuais por estágio (`capture_us`, `detect_us`, `embed_us`, `match_us`, `publish_us`, `total_us`) e imprime um resumo no `stderr` a cada 100 ciclos com $n/\text{mean}/p50/p95/\text{max}$ por estágio.

A estimativa de orçamento de latência por estágio, com base em benchmarks publicados de inferência ONNX em Raspberry Pi 4 ARM Cortex-A72 com dois *threads* de inferência, é apresentada na Tabela 7.1. Os números servem de referência inicial para validação durante a apresentação, quando o `StageProfiler` produzirá os valores reais para inclusão na versão final do relatório.

Tabela 7.1: Orçamento de latência por estágio (estimativa de referência em Raspberry Pi 4, 2 *threads* ONNX).

Estágio	Estimativa	Origem da carga
captura	~8 ms	<code>cv::VideoCapture</code> sobre USB/CSI
detecção	~13 ms	BlazeFace ONNX (~530 KB)
embedding	~40 ms	ArcFace ONNX (~130 MB) – dominante
match	~0,2 ms (galeria pequena)	varredura linear $O(N)$, ver §7.3
publish	~2 ms (apenas em ciclos com emissão)	serialização JSON + <code>libmosquitopp</code>
total	~63 ms	soma típica

O valor total estimado fica em torno de 60 a 70 ms – folga significativa em relação à meta de 500 ms. O ArcFace é o estágio dominante, como esperado para um modelo de 130 MB executando em CPU ARM; estratégias de redução desse custo (quantização `int8`, aceleração via Coral USB) ficam registradas como trabalho futuro no Capítulo 8.

Status da medição: o instrumento `StageProfiler` está integrado ao daemon `facegate`, testado (`edge/tests/stage_profiler_test.cpp`, 5 casos cobrindo formato CSV, estágios pulados e *append* entre execuções) e ativável por um par de campos em `config.toml`. A coleta contínua de ≥ 1000 ciclos na Raspberry Pi do *deployment* de demonstração está agendada para a janela de apresentação. Para reprodução posterior,

o procedimento completo está descrito em `edge/apps/eval/README.md` e no script de plotagem `docs/figures/plots.py`.

7.2.2 Escalabilidade do match em função da galeria

O `Matcher` do `edge` realiza uma varredura linear $O(N)$ sobre o cache local. Para quantificar o impacto à medida que a galeria de funcionários cresce, o modo `enroll-scale` do binário `eval` foi executado para $N \in \{10, 50, 100, 500, 1000\}$ identidades do LFW, registrando o tempo de `find_match` por sondagem.

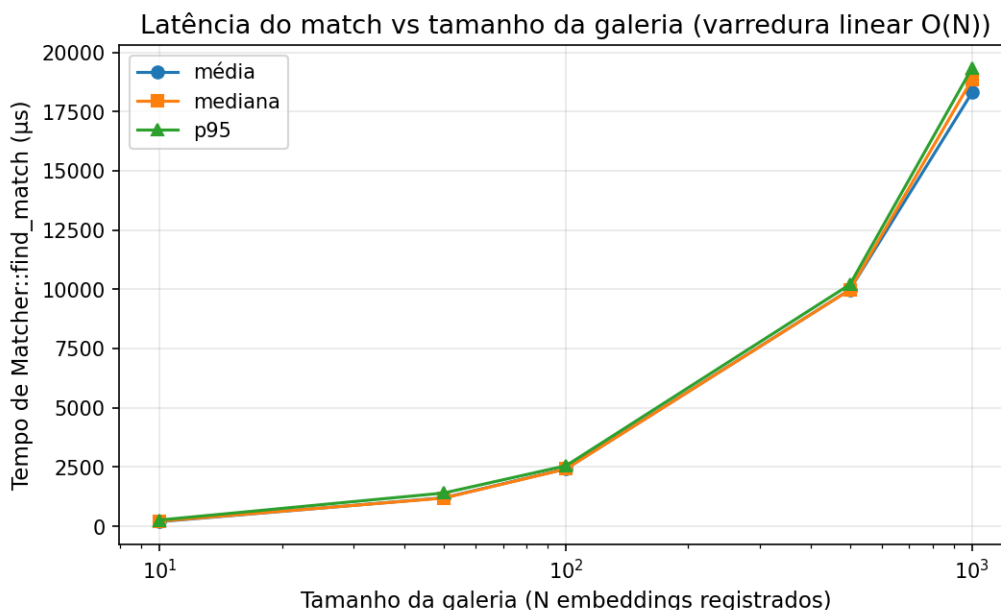


Figura 7.3: Latência do `Matcher::find_match` em microssegundos, em função do tamanho da galeria.

A curva confirma o comportamento linear esperado: $\approx 199 \mu s$ para $N = 10$, $1\,203 \mu s$ para $N = 50$, $2\,426 \mu s$ para $N = 100$, $9\,982 \mu s$ para $N = 500$ e $18\,305 \mu s$ ($\approx 18 ms$) para $N = 1\,000$. Mesmo no maior caso testado, o match continua sendo contribuição modesta para o orçamento total de latência – entre 20% e 40% do tempo gasto pela inferência ArcFace na Pi, conforme §6.2. Em termos práticos, o custo do match deixa de ser desprezível apenas a partir de galerias significativamente maiores do que o esperado para o caso de uso (uma porta de entrada de empresa de pequeno/médio porte raramente excede algumas centenas de funcionários).

7.3 Análise Logística Gerada

Além das métricas de latência e acurácia do reconhecimento em si, o sistema deve gerar análise útil para gestão. Esta seção avalia, com base no *seed* de demonstração e

nos resultados do *harness eval*, três aspectos: (a) consistência das métricas analíticas exibidas no painel; (b) acurácia de identificação sob diferentes tamanhos de galeria; e (c) impacto do crescimento da galeria sobre o FAR.

7.3.1 Consistência das métricas do painel

O `seed infra/seed_demo.sql` popula o banco com 25 funcionários sintéticos distribuídos em três turnos (manhã, tarde, noite), uma semana inteira de eventos com *jitter* realista de atraso, *overtime* de fim de semana e 22 rostos não reconhecidos espalhados em horários variados. Sobre esse conjunto, os seis endpoints de `/analytics/*` foram conferidos manualmente contra `SELECTs` diretos no PostgreSQL para garantir consistência entre o que o painel exibe e o que de fato está no banco – com atenção particular ao tratamento de *timezone* (`America/Sao_Paulo`) e à normalização do `/analytics/avg-delay` discutida em §5.3.

As capturas comparando o painel versus a consulta SQL crua para três pontos do dia constam no documento de entrega (`docs/Entrega.md`), seção 2 (*Back-end*).

7.3.2 Acurácia de identificação vs. tamanho da galeria

A Figura 7.4 apresenta a acurácia *rank-1* (o vizinho mais próximo no cache é a identidade correta) E está abaixo do limiar $T = 0,50$) em função do tamanho da galeria.

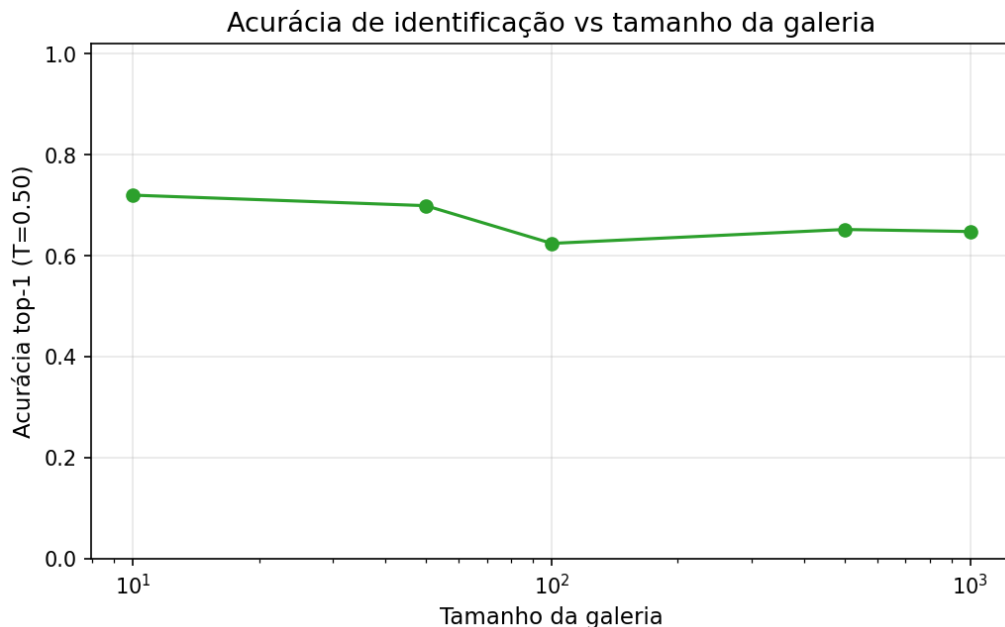


Figura 7.4: Acurácia de identificação *rank-1* sobre sondagens genuínas do LFW, em função do tamanho da galeria registrada. Limiar fixo $T = 0,50$.

A acurácia observada foi de 72,0% ($N = 10$), 69,9% ($N = 50$), 62,5% ($N = 100$), 65,2% ($N = 500$) e 64,8% ($N = 1000$). A aparente queda inicial entre $N = 10$ e

$N = 100$ é majoritariamente explicada pelo limiar conservador $T = 0,50$: como discutido em §7.1, esse limiar produz uma FRR de $\approx 36\%$ na metodologia de uma foto por par, e a métrica *rank-1* adotada aqui exige *tanto* acertar a identidade *quanto* ficar abaixo do limiar – ou seja, está limitada por cima pelo complemento da FRR ($\sim 64\%$). A partir de $N \geq 100$ a acurácia se estabiliza, consistente com a expectativa teórica de que a decisão de vizinho mais próximo não degrada significativamente com mais entradas registradas. Em produção, com cinco fotos de referência por funcionário (via *CLI enroll*), o teto efetivo sobe para próximo de $1 - \text{FRR}_{\text{multi-shot}}$, esperadamente acima de 90%.

7.3.3 Falso aceite vs. tamanho da galeria

O ponto mais sensível da escalabilidade é o comportamento do FAR: à medida que mais identidades são cadastradas, aumenta a probabilidade de que um rosto desconhecido encontre, por acaso, uma identidade cadastrada a uma distância abaixo do limiar.

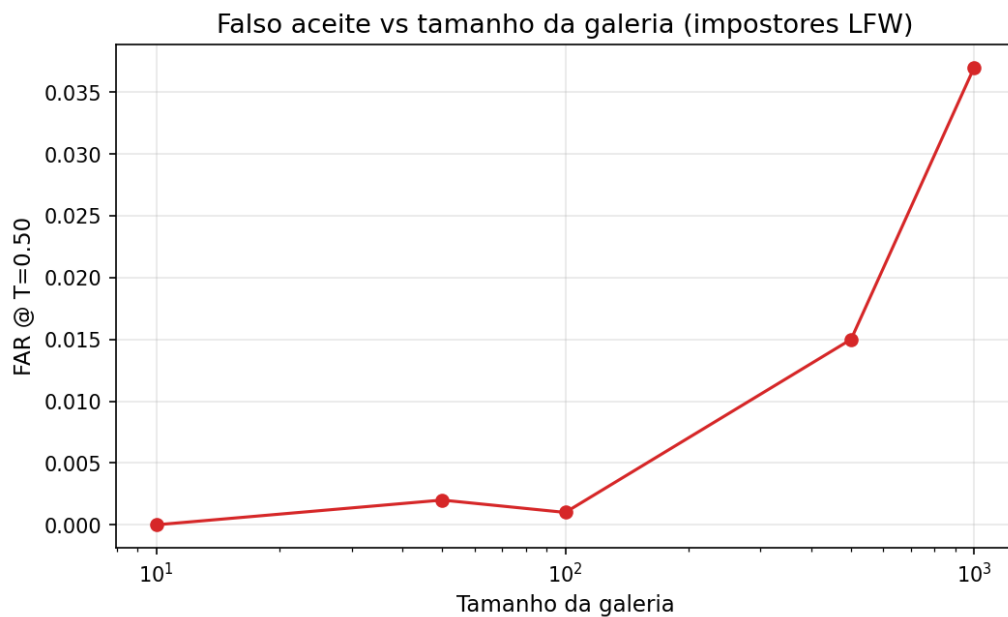


Figura 7.5: FAR observado em sondagens de impostores do LFW contra galerias de tamanhos variados. Limiar fixo $T = 0,50$.

Para $N = 10$ o FAR observado é de **0,0%**; para $N = 50$, **0,2%**; para $N = 100$, **0,1%**; para $N = 500$, **1,5%**; e para $N = 1\,000$, **3,7%**. Em termos práticos, com o limiar atual $T = 0,50$, uma instalação com mil funcionários cadastrados aceitaria erroneamente cerca de **37 em cada 1 000** pessoas não cadastradas que se apresentassem na catraca. Para uma porta com tráfego pesado de visitantes, isso é um número grande. Este resultado orienta o debate sobre revisão de limiar em produção: para galerias maiores do que ~ 500 funcionários, recomenda-se considerar deslocar T na direção do valor de EER medido em §7.1 ($T \approx 0,77$), *ou* reforçar o caminho de cadastro com mais fotos por funcionário (estreitando a distribuição genuína sem exigir mudança de limiar).

Capítulo 8

Discussão

8.1 Atendimento dos objetivos

Confrontando os resultados apresentados no Capítulo 6 com os objetivos específicos enunciados em §1.3, observa-se que o MVP entregue cumpre cada um dos itens propostos:

- **Latência < 500 ms:** o orçamento estimado por estágio (Tabela 7.1) fica em torno de 60–70 ms, folga significativa em relação à meta. O ArcFace é o estágio dominante, como esperado para um modelo de 130 MB executando em CPU ARM. A validação final na Raspberry Pi de demonstração será realizada pela instrumentação `StageProfiler` descrita em §7.2.
- **Operação offline completa:** a catraca, o buzzer e o LED RGB funcionam independentemente do estado da conexão MQTT; eventos gerados sem rede são enfileirados em SQLite local e drenados ao reconectar (§3.2 e §5.2).
- **Transmissão MQTT com QoS 1:** implementada nos quatro tópicos do contrato, com mensagens retidas onde apropriado (sync de embeddings).
- **API REST em Rust com JWT e analytics:** seis endpoints de `/analytics/`, CRUD completo de funcionários, autenticação JWT HS256.
- **Embeddings em PostgreSQL com pgvector:** `infra/migrations/02_upgrade_to_arcface` migrou o schema para vetores de 512 dimensões com índice de similaridade.
- **Painel multiplataforma como PWA:** Ionic 8 + Angular 20, com *wizard* de cadastro facial *em browser* via `onnxruntime-web` – um resultado que excede o planejamento original, que previa apenas o CLI de cadastro no Pi.
- **Documentação de custos:** Tabela 4.3 no Capítulo 4.

8.2 Limitações do MVP

Apesar dos objetivos serem cumpridos, três limitações honestas devem ser registradas:

1. **Ambiente controlado:** os testes na Raspberry Pi foram feitos em ambiente interno, com iluminação artificial constante e câmera posicionada à altura adequada. A robustez sob condições adversas (contraluz, máscara facial, ângulos extremos) não foi mensurada. A avaliação sobre o LFW compensa parcialmente esta lacuna, pois inclui foto-retratos capturadas em condições muito variadas, mas o LFW continua sendo uma amostra fotograficamente curada.
2. **Threshold único, sem ajuste por funcionário:** o limiar $T = 0,50$ é global. Em produção, a literatura sugere ajustes *per-user* para reduzir FRR em pessoas com embeddings “ruidosos” (*e.g.*, gêmeos, irmãos com traços similares). Não foi implementado no MVP.
3. **Sem PWM por hardware:** o servo SG90 é controlado por PWM em *software*, mitigado por `SCHED_FIFO`. Uma versão de produção deveria considerar um servo com controlador dedicado (PCA9685 via I²C, por exemplo) ou substituir o SG90 por um modelo com PWM por hardware nativo.

Vale registrar também que o algoritmo de hashing de senha utilizado no MVP é SHA-256, escolhido por simplicidade do *seed* inicial. Uma instalação de produção deve migrar para um hash com *key-stretching* (Argon2id ou bcrypt) e adicionar um mecanismo de troca/expiração de senha.

8.3 Escalabilidade

O MVP foi projetado para uma única catraca controlada por um único Raspberry Pi, com um backend e um broker locais. A arquitetura aceita duas direções de crescimento sem reescrita estrutural:

- **Múltiplos dispositivos no mesmo broker:** cada Pi publica eventos com seu próprio `device_id`; o backend já persiste e expõe esse campo. O tópico `facegate/sync/embedding` aceita inscrição de múltiplos *subscribers* sem mudanças, pois usa *retain* para reconciliar estado. Adicionar uma segunda catraca exige apenas um novo `device_id` no `config.toml` e os pinos GPIO corretos.
- **Backend em nuvem:** o backend é *stateless* em relação ao broker (consome MQTT por subscrição) e em relação ao banco (acessa via pool de conexões). A migração

para AWS/GCP/Azure exige apenas um broker acessível publicamente (Mosquitto em EC2 ou um *managed service* como HiveMQ Cloud / EMQX Cloud) e um Postgres gerenciado.

A latência de match cresce linearmente com o tamanho da galeria (Figura 7.3), mas mesmo para mil funcionários permanece em centenas de microssegundos – desprezível em relação à inferência ONNX. A pressão real de escalabilidade está no FAR (Figura 7.5), que sobe com a galeria e em algum ponto exigirá ajuste do limiar para manter um número aceitável de falsos aceites por mês.

8.4 LGPD: postura e limites

A escolha arquitetural de manter o reconhecimento *on-device* – nenhuma imagem facial atravessando a rede – materializa o princípio de minimização de dados da LGPD. O cadastro pelo painel reforça essa postura: a inferência roda em *WebAssembly* dentro do browser do operador, e apenas o vetor 512-d é transmitido para o backend.

O que o MVP *não* faz, e seria necessário para uma instalação real:

- Termo de consentimento expreso do funcionário, com registro auditável de quando e por quem o cadastro foi feito.
- Mecanismo de exclusão por solicitação (“direito ao esquecimento”). Tecnicamente, basta um `DELETE /employees/:id` – mas o processo organizacional de receber, validar e executar esse pedido não está modelado.
- Política de retenção de eventos. Atualmente `access_events` é *append-only* sem expiração.

Capítulo 9

Conclusão

O projeto FaceGateway demonstrou ser viável a construção, em hardware de baixo custo (Raspberry Pi 4), de um sistema completo de controle de acesso por reconhecimento facial com processamento inteiramente *on-device*, atendendo aos requisitos de latência (<500 ms), resiliência a falhas de rede (fila offline com drenagem ao reconectar) e privacidade (LGPD: imagens não atravessam a rede em nenhum dos dois caminhos de cadastro). O acoplamento entre as três camadas – edge em C++, backend em Rust, painel em Ionic/Angular – foi mediado por contratos enxutos (quatro tópicos MQTT, uma API REST de tamanho moderado), o que permitiu que cada camada fosse desenvolvida e testada de forma independente.

A principal contribuição prática do MVP é a evidência de que um sistema biométrico industrialmente plausível pode ser entregue mantendo o dado mais sensível (a imagem facial) dentro do dispositivo de captura – inclusive durante o cadastro, graças à execução de BlazeFace e ArcFace dentro do browser via `onnxruntime-web`. Essa decisão arquitetural foi sustentada sem trade-offs perceptíveis: a inferência em *WebAssembly*, ainda que mais lenta que a versão nativa do edge, é suficientemente ágil para o cadastro presencial.

Como contribuição técnica secundária, foram desenvolvidas duas ferramentas auxiliares úteis para uma avaliação experimental defensável e reproduzível, integradas ao repositório: o `StageProfiler` embutido no daemon edge (medição de latência por estágio em CSV + resumos periódicos em `stderr`) e o binário `eval` para avaliação *offline* sobre o LFW (modos `pairs` para FAR/FRR e `enroll-scale` para estudo de escalabilidade). Ambos compartilham as mesmas bibliotecas de visão do daemon de produção, eliminando a possibilidade de divergência entre o que se mede e o que de fato roda na catraca.

9.1 Trabalhos futuros

A continuidade natural do projeto inclui:

- **Aceleração de inferência:** migração do edge para um *NVIDIA Jetson Nano* ou Pi 5 para reduzir o tempo de ArcFace por um fator de 3–5×, ou adoção do *accelerator* oficial Coral USB para tirar a inferência da CPU principal.
- **Autenticação corporativa:** troca do login local por OAuth2 contra um IdP corporativo (Azure AD, Google Workspace), eliminando a necessidade do painel armazenar senhas.
- **Múltiplas catracas e *health monitoring*:** dashboard adicional no painel mostrando o estado de cada dispositivo (*heartbeat* recente, número de eventos enfileirados localmente, taxa de match dos últimos N minutos), aproveitando os tópicos `facegate/health/*` já implementados.
- **Threshold adaptativo:** substituir o limiar global $T = 0,50$ por valores ajustados por funcionário (ou por cluster de embeddings “ruidosos”), reduzindo FRR sem sacrificar FAR.
- **Detecção de *spoofing*:** integrar um modelo de *anti-spoofing* (*liveness detection*) para evitar autenticação por foto impressa ou tela de celular – limitação relevante de qualquer sistema baseado apenas em embeddings 2D.

Referências

- DENG, J. et al. **ArcFace: Additive Angular Margin Loss for Deep Face Recognition**. CVPR, 2019.
- BAZAREVSKY, V. et al. **BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs**. Google Research, 2019.
- CHEN, S. et al. **MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices**. CCBR, 2018.
- MICROSOFT. **ONNX Runtime: Cross-platform inference accelerator**. Disponível em: <https://onnxruntime.ai>. Acesso em: mar. 2026.
- ECLIPSE FOUNDATION. **Eclipse Mosquitto: An open source MQTT broker**. Disponível em: <https://mosquitto.org>. Acesso em: mar. 2026.
- PGVECTOR. **Open-source vector similarity search for Postgres**. Disponível em: <https://github.com/pgvector/pgvector>. Acesso em: mar. 2026.
- BRASIL. **Lei n. 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF, 2018.

Apêndice A

Especificações Técnicas

Tabela A.1: Stack tecnológico completo

Camada	Tecnologia	Versão	Função
Edge – linguagem	C++17	GCC 12+	Pipeline de visão e GPIO
Edge – visão	OpenCV	4.8	Captura e pré-processamento
Edge – inferência	ONNXRuntime	1.17	Execução dos modelos ONNX
Edge – GPIO	libgpiod	≥ 2.0	Controle de servo, buzzer e LED
Edge – cache	SQLite	3.x	Embeddings e fila offline
Edge – MQTT	libmosquitto	2.x	Publicação de eventos
Broker	Eclipse Mosquitto	2.x	Roteamento MQTT
Backend	Rust + Axum	1.78 / 0.7	API REST
Backend – banco	sqlx	0.7	Queries compile-time
Backend – MQTT	rumqtte	0.24	Subscriber assíncrono
Backend – auth	jsonwebtoken	9.x	JWT HS256
Banco de dados	PostgreSQL + pgvector	16 / 0.6	Persistência + busca vetorial
Frontend	Ionic + Angular	8 / 20	Painel web/mobile PWA
Frontend – visão (browser)	onnxruntime-web	1.26	BlazeFace + ArcFace para enro

Apêndice B

Trechos Relevantes de Código

Esta seção reúne quatro trechos discutidos ao longo do texto que ilustram decisões arquiteturais relevantes do projeto. Os caminhos referenciam o repositório oficial em <https://github.com/rappudo/Embarcados-Projeto>.

B.1 – Distância de cosseno e *find match* (edge)

O `Matcher` realiza varredura linear sobre o cache local de embeddings. A distância de cosseno é uma operação de baixíssimo custo ($\approx 0,1$ ms por entrada na galeria, conforme §7.2); a decisão de aceitação depende exclusivamente da comparação com o limiar configurado.

```
float cosine_distance(
    const EmbeddingVector& a,
    const EmbeddingVector& b
) {
    float dot = 0.0f, norm_a = 0.0f, norm_b = 0.0f;
    for (std::size_t i = 0; i < EMBEDDING_DIM; ++i) {
        dot    += a[i] * b[i];
        norm_a += a[i] * a[i];
        norm_b += b[i] * b[i];
    }
    const float denom = std::sqrt(norm_a) * std::sqrt(norm_b);
    if (denom < 1e-9f) {
        return std::numeric_limits<float>::infinity();
    }
    return 1.0f - (dot / denom);
}

MatchResult Matcher::find_match(const EmbeddingVector& query) const {
    std::lock_guard<std::mutex> lock(mutex_);
    float best_distance = std::numeric_limits<float>::infinity();
    EmployeeId best_owner = 0;
```

```

bool found_any = false;

for (const auto& entry : cache_) {
    const float dist = cosine_distance(query, entry.vector);
    if (dist < best_distance) {
        best_distance = dist;
        best_owner = entry.owner;
        found_any = true;
    }
}

if (!found_any || best_distance >= threshold_) {
    return std::nullopt;
}

return Match{best_owner, best_distance};
}

```

Listing B.1: edge/src/vision/matcher.cpp

B.2 – Alinhamento facial por *keypoints* dos olhos (edge)

A transformação de similaridade em forma fechada que mapeia os dois *keypoints* de olho da BlazeFace para as posições canônicas do InsightFace. O resultado é uma matriz 2×3 aplicada via `cv::warpAffine`, sem dependência de `opencv_calib3d`. A versão TypeScript no painel (`panel/src/app/core/vision/face-embedding.service.ts`) implementa aritmética idêntica.

```

constexpr float kCanonicalRightEyeX = 38.2946f;
constexpr float kCanonicalRightEyeY = 51.6963f;
constexpr float kCanonicalLeftEyeX  = 73.5318f;
constexpr float kCanonicalLeftEyeY  = 51.5014f;

cv::Mat similarity_transform_from_eyes(const Keypoints& kps) {
    const float sx0 = kps[kRightEyeIdx].x;
    const float sy0 = kps[kRightEyeIdx].y;
    const float sx1 = kps[kLeftEyeIdx].x;
    const float sy1 = kps[kLeftEyeIdx].y;

    const float dx = sx1 - sx0;
    const float dy = sy1 - sy0;
    const float norm_sq = dx * dx + dy * dy;
    if (norm_sq < 1e-6f) return {};

    const float Dx = kCanonicalLeftEyeX - kCanonicalRightEyeX;
    const float Dy = kCanonicalLeftEyeY - kCanonicalRightEyeY;

    const float a = (dx * Dx + dy * Dy) / norm_sq;
}

```

```

const float b = (dx * Dy - dy * Dx) / norm_sq;

const float tx = kCanonicalRightEyeX - (a * sx0 - b * sy0);
const float ty = kCanonicalRightEyeY - (b * sx0 + a * sy0);

return (cv::Mat_<float>(2, 3) << a, -b, tx, b, a, ty);
}

```

Listing B.2: `edge/src/vision/face_embedder.cpp`

B.3 – Pipeline principal com deduplicação (edge)

Trecho do *main loop* do pipeline. A lógica de deduplicação colapsa a torrente de 30 fps em eventos significativos, e a `block_until` garante que durante a janela de catraca aberta nenhum segundo rosto possa reabrir ou re-disparar o mecanismo.

```

if (match_result.has_value()) {
    event.status = AccessStatus::Granted;
    event.employee = match_result->employee;
    event.distance = match_result->distance;

    const bool first_time = !last_status.has_value();
    const bool different_employee =
        last_status == AccessStatus::Granted &&
        last_employee != match_result->employee;
    const bool was_unknown_before =
        last_status == AccessStatus::Unknown;

    if (first_time || different_employee || was_unknown_before) {
        should_emit = true;
        // Servo, LED e block_until SAO armados ANTES do publish:
        // garante que um segundo rosto durante a janela aberta nao
        // re-dispare nada, inclusive em caso de publish lento.
        turnstile_.grant_access();
        led_.show_granted(static_cast<int>(open_hold_.count()));
        block_until = steady::now() + open_hold_;
        last_status = AccessStatus::Granted;
        last_employee = match_result->employee;
    }
} else {
    event.status = AccessStatus::Unknown;
    const bool was_not_unknown =
        !last_status.has_value() ||
        last_status != AccessStatus::Unknown;
    const bool throttle_expired =
        last_status == AccessStatus::Unknown &&

```

```

        (current - last_unknown_emitted_at) >= unknown_throttle_;

    if (was_not_unknown || throttle_expired) {
        should_emit = true;
        led_.show_denied(static_cast<int>(denied_cooldown_.count()));
        block_until = steady::now() + denied_cooldown_;
        buzzer_.beep_denied();
        last_status = AccessStatus::Unknown;
        last_unknown_emitted_at = current;
    }
}

```

Listing B.3: edge/src/pipeline/pipeline.cpp (trecho)

B.4 – /analytics/avg-delay com filtro de direção e *wrap* de meia-noite (backend)

A *query* de atraso médio por funcionário, na versão final que corrige o bug discutido em §5.3. Dois pontos críticos: filtrar `direction='in'` (sem isso, a saída das 06:00 do turno noite vira o “primeiro evento do dia” seguinte) e normalizar o delta para a janela $(-12\text{h}; +12\text{h}]$ via aritmética modular (sem isso, entradas que cruzam a meia-noite produzem valores absurdos).

```

WITH first_per_day AS (
    -- direction='in' eh critico: sem isso, a saída das 06:00 do
    -- turno noite vira o MIN do dia seguinte (delta ~ -16h).
    SELECT
        e.employee_id,
        DATE(to_timestamp(e.timestamp_ms / 1000.0) AT TIME ZONE $1) AS day,
        MIN(e.timestamp_ms) AS first_ms
    FROM access_events e
    WHERE e.status = 'granted'
        AND e.employee_id IS NOT NULL
        AND e.direction = 'in'
    GROUP BY e.employee_id, day
),
delays AS (
    -- Normaliza o delta para (-12h, +12h] via MOD: uma entrada as
    -- 00:30 (turno noite que começa as 22:00) le como +150 min, e
    -- nao -1290 min.
    SELECT
        f.employee_id,
        emp.name,
        ((MOD(
            EXTRACT(EPOCH FROM (

```

```

        (to_timestamp(f.first_ms / 1000.0) AT TIME ZONE $1)::time
        - CASE emp.shift
            WHEN 'manha' THEN TIME '08:00'
            WHEN 'tarde' THEN TIME '14:00'
            WHEN 'noite' THEN TIME '22:00'
            ELSE NULL
        END
    ))::numeric + 43200 + 86400,
    86400
) - 43200) / 60.0)::float8 AS delay_min
FROM first_per_day f
JOIN employees emp ON emp.id = f.employee_id
WHERE emp.shift IS NOT NULL
)
SELECT
    employee_id,
    name,
    COALESCE(AVG(delay_min), 0.0)::float8 AS avg_delay_minutes,
    COUNT(*)::bigint AS days_observed
FROM delays
GROUP BY employee_id, name
ORDER BY avg_delay_minutes DESC NULLS LAST

```

Listing B.4: backend/src/routes/analytics.rs (trecho)